

Računarska vežba 9

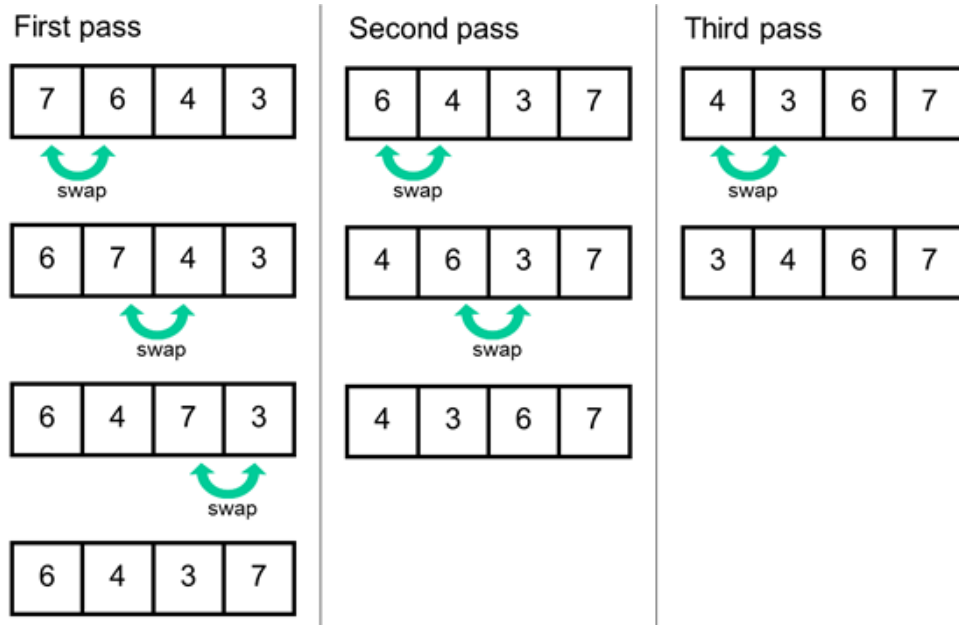
PYTHON ALGORITMI SORTIRANJA

1 Algoritam Bubble Sort

Ideja funkcionisanja Bubble Sort algoritma je jednostavna: za datu neuređenu listu ili niz, poredimo susedne elemente i nakon svakog poređenja ih postavljamo u ispravan redosled prema njihovim vrednostima tj. menjamo susedne elemente ako nisu u ispravnom redosledu i ovaj proces se ponavlja $(n-1)$ puta za listu od n elemenata

U svakoj iteraciji ili prolazu (pass), najveći element liste se pomera na kraj liste. Nakon druge iteracije, drugi najveći element će biti postavljen na preposlednju poziciju na listi i isti postupak se ponavlja dok se niz ne sortira.

Primer sortiranja liste 7, 6, 4, 3



Iako jednostavan, prosečno vreme izvršavanja ovog algoritma je $O(n^2)$ što ga čini neprikladnim pri radu sa velikim ulaznim podacima (nizovima ili listama).

Bubble Sort algoritam sortiranja počinje sa spoljašnjom petljom koja iterira kroz niz da bi se izvršilo više prolaza. U svakom prolazu, unutrašnja petlja upoređuje susedne elemente i zamenjuje ih ako su u pogrešnom redosledu. Do kraja svakog prolaza kroz niz, najveći element će da „istrči“ do svoje ispravne pozicije na kraju niza.

Spoljašnja petlja (**i**) se kreće od **0** do **n-2** jer nakon što se prođe **n-1** elemenata, najveći element je na svojoj ispravnoj poziciji-

Unutrašnja petlja (**j**) se kreće od **0** do **n-i-2** jer je najvećih **i** elemenata već sortirano na kraju niza nakon svakog prolaza. Ako je trenutni element niza [**j**] veći od sledećeg elementa niza [**j+1**], vrši se zamena. Proces se nastavlja dok se ceo niz ne sortira i više nisu potrebne zamene

Programski kôd:

```
1  # Bubble sort algoritam....
2
3  def bubble_sort(array):
4      n = len(array)
5      for i in range(n-1):
6          for j in range(n-i-1):
7              if array[j] > array[j+1]:
8                  array[j], array[j+1] = array[j+1], array[j]
9      return array
10
11 arr = [10, 5, 2, 8, 3]
12 print(f"Input: arr = {arr}")
13 print("Output:", bubble_sort(arr))
14 |
```

Objašnjenje koda:

```
# Bubble sort algoritam....
```

- Komentar...

```
def bubble_sort(array):
```

- Definisanje funkcije `bubble_sort` sa jednim argumentom `array`
- Po konvenciji `array` je promenljiva sekvenca (mutable) kao i Python lista, sa elementima koji se mogu porediti (numbers, strings, etc.).

```
    n = len(array)
```

- Izračunava se dužina ulazne liste preko `len(array)` i čuva se u lokalnoj promenljivoj `n`.
- `n` se koristi za kontrolu granica obe petlje kako se ne bi prešle granice liste

```
    for i in range(n-1):
```

- Start spoljašnje petlje, koja se izvršava `n-1` puta: `for i = 0, 1, ..., n-2`
- Svaka iteracija kroz ovu petlju znači jedan prolaz kroz listu, tokom koje se najveći element postavlja na kraj kao najveći
- Uzima se `n-1` umesto `n` jer nakon `n-1` prolaza, niz će sigurno biti sortiran

```
        for j in range(n-i-1):
```

- Start unutrašnje petlje, kojom se porede susedni element i i zamenjuju se ako nisu u pravom redosledu. Opseg `range(n-i-1)` generiše prolazak od `0` do `n-i-2`, pri čemu izraz `n-i-1` smanjuje opseg pri svakom prolazu jer poslednji `i` element je na poslednjoj sortiranoj poziciji i ne mora ponovo da se poredi. Na ovaj način se izbegava nepotrebno poređenje i delimično poboljšavaju performanse u odnosu na iteraciju do `n-1`

```
            if array[j] > array[j+1]:
```

- Porede se elementi na pozicijama sa indeksima `j` i `j+1`.

- Ako je levi element veći od desnog, oni menjaju mesta,

```
array[j], array[j+1] = array[j+1], array[j]
```

- Izvršava se zamena dva susedna elementa upotrebom tuple raspakivanja
- Na desnoj strani se prvo definiše privremeni par vrednosti (`array[j+1]`, `array[j]`), na levoj strani se ove dve vrednosti dodeljuju vrednostima `array[j]` and `array[j+1]`, menjajući njihove pozicije
- Pošto se to dešava in-place, u istom memorijskom prostoru, originalna lista `array` se direktno modifikuje pa se ne kreira nova lista tokom sortiranja

```
return array
```

- Nakon spoljašnje petlje, prekida se sa prolazima i lista je sortirana u rastućem redosledu
- Funkcija vraća objekte liste koji su sortirani (`sorted_arr = bubble_sort(arr)`).

```
arr = [10, 5, 2, 8, 3]
```

- Definiše se lista pod imenom `arr` sa 5 celobrojnih elemenata koji su nesortirani

```
print(f"Input: arr = {arr}")
```

- Koristi se f-string (formatted string literal) za print liste `arr`.
- Izraz `{arr}` unutar stringa se prvo evaluira i konvertuje u string formu (tj. `[10, 5, 2, 8, 3]`), i interpolira se na izlaz, lista `arr` je još uvek nesortirana jer funkcija `bubble_sort` još nije pozvana

```
print("Output:", bubble_sort(arr))
```

- Poziv `bubble_sort` funkcije sa `arr` kao argumentom, započinje proces sortiranja
- Return (sortirane) liste se štampa u okviru "Output: ", pa konzola prikazuje sortiranu listu `arr`, kao `[2, 3, 5, 8, 10]`.

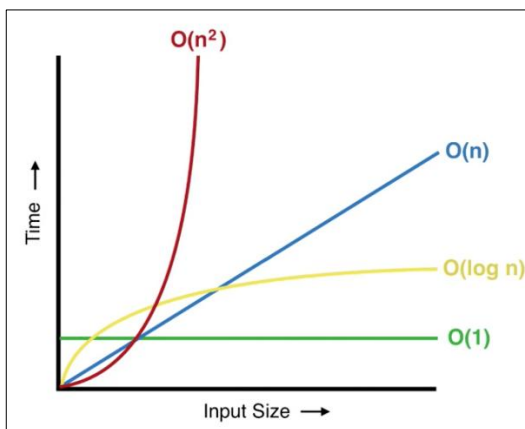
Jednostavniji kôd:

```

1  def bubble_sort(arr):
2      n = len(arr)
3      for i in range(n):
4          for j in range(0, n - i - 1):
5              if arr[j] > arr[j + 1]:
6                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
7      return arr
8
9  data = [64, 34, 25, 12, 22, 11, 90]
10 print(bubble_sort(data.copy()))

```

● [11, 12, 22, 25, 34, 64, 90]



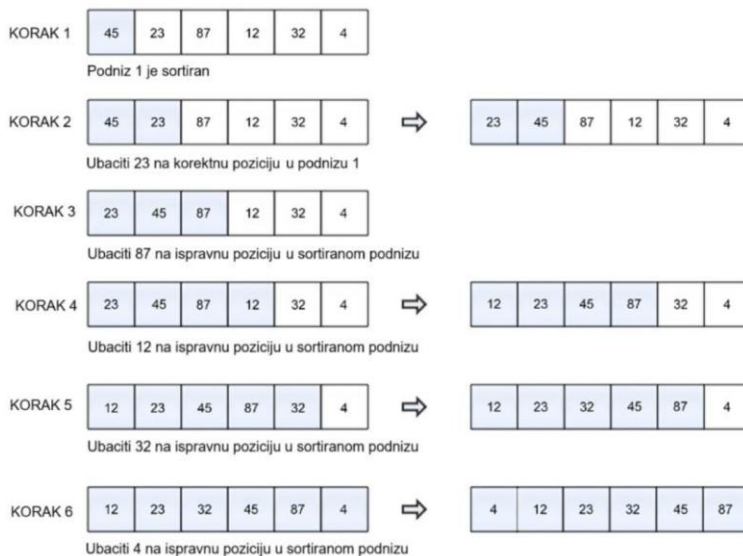
2 Algoritam Insertion Sort

Insertion sort je algoritam za sortiranje sa prosečnom vremenoskom složenošću $O(n^2)$

Pogodan je samo za sortiranje manjih nizova ili listi ulaznih podataka. Dizajniran je za efikasno sortiranje elemenata postepenim formiranjem sortiranog podniza unutar nesortiranog niza. Glavna ideja ovog algoritma je ispitati svaki nesortirani element i umetnuti ga na njegovu ispravnu poziciju unutar sortiranog podniza, čime se sortirani podniz postupno povećava.

Algoritam formira dve podliste (podniza), jednu koja je sortirana i jednu koja nije sortirana, u kojima se elementi dodaju jedan po jedan iz nesortirane podliste u sortiranu. Uzimaju se elementi iz nesortirane podliste i ubacuju na ispravnu poziciju u sortiranoj podlisti.

PRIMER 1: Sortiranje niza od 6 elemenata [45, 23, 87, 12, 32, 4]



- Počinje se od prvog elementa, pod pretpostavkom da je sortiran
- Uzima se sledeći element, 23, iz nesortiranog podniza i ubacuje se na ispravnu poziciju u sortiranom podnizu
- U sledećoj iteraciji uzima se treći element 87 iz nesortirane podliste i ubacuje se u sortirani podniz na ispravnu poziciju
- Ponavljaju se iste iteracije sve dok se svi elementi ne nađu u sortiranoj podlisti

Insertion Sort je jednostavan algoritam sortiranja zasnovan na poređenju koji formira sortirani niz iterativnim umetanjem svakog elementa na njegovu odgovarajuću poziciju unutar već sortiranog dela niza. Algoritam počinje sa spoljašnjom petljom koja iterira od drugog do poslednjeg elementa niza, razmatra trenutni element (**array[i]**) i pomera ga na ispravnu poziciju unutar sortiranog dela niza. Unutrašnja while petlja počinje od indeksa tekućeg elementa (**j=i**) i nastavlja ka početku niza (**j>0**) sve dok je tekući element manji od susednog (**array[j]<array[j-1]**). U svakoj iteraciji unutrašnje petlje, ako je uslov zadovoljen, vrši se zamena da bi se tekući element pomerio za jednu poziciju unazad. Proces se nastavlja sve dok tekući element ne bude na ispravnoj poziciji, a zatim se spoljašnja petlja pomera na sledeći element.

Programski kôd 1:

```

1  # Insertion Sort algoritam....
2
3  def insertion_sort(array):
4      for i in range(1, len(array)):
5          j = i
6
7          while j > 0 and array[j] < array[j-1]:
8              array[j], array[j-1] = array[j-1], array[j]
9              j -= 1
10
11     return array
12
13 arr = [10, 5, 2, 8, 3]
14 print(f"Input: arr = {arr}")
15 print("Output:", insertion_sort(arr))

```



- Input: arr = [10, 5, 2, 8, 3]
Output: [2, 3, 5, 8, 10]

Programski kôd 2:

```
# Funkcija za sortiranje Insertion sort
def insertionSort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Pomeranje elemenata niza arr[0..i-1], koji su
        # veći od Key, za jednu poziciju iza tekuće
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

# Funkcija za ispis niza veličine n
def printArray(arr):
    for i in range(len(arr)):
        print(arr[i], end=" ")
    print()

# Driver kod
if __name__ == "__main__":
    arr = [12, 11, 13, 5, 6]
    insertionSort(arr)
    printArray(arr)
```

Output

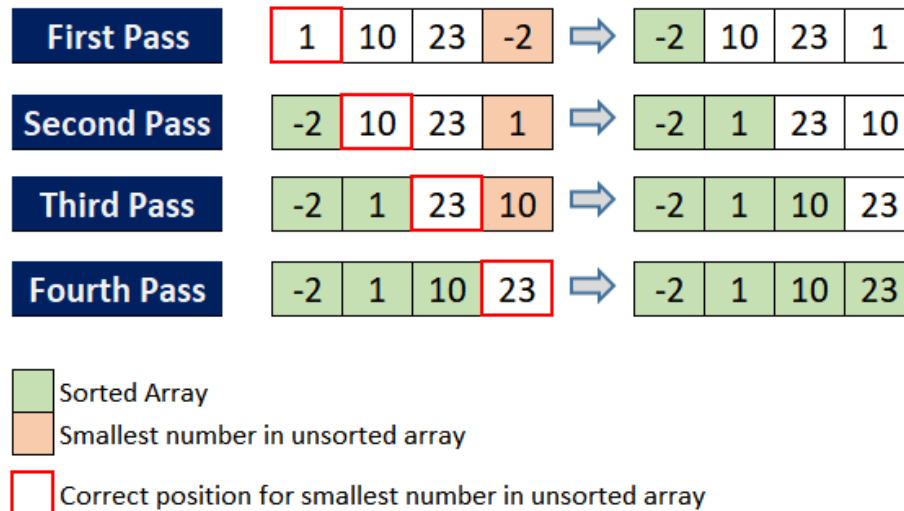
5 6 11 12 13

3 Algoritam Selection Sort

Za razliku od algoritma Bubble Sort koji postavlja najveće elemente na kraj niza, Selection Sort radi obrnuto, postavlja najmanje elemente na početak niza. U svakom prolazu trenutni element niza se deklarise kao najmanji, a zatim se u ostatku niza sa desne strane traži element sa manjom vrednošću. Ukoliko je najmanji element sa desne strane niza manji od trenutnog elementa niza, vrši se njihova zamena (swap).

Vremenska složenost izvršavanja ovog algoritma je **$O(n^2)$**

Iako algoritmi Bubble Sort i Selection Sort imaju istu vremensku složenost $O(n^2)$, Bubble Sort zahteva više zamena nego Selection Sort, jer u svakom prolazu može zameniti susedne elemente više puta dok se najveći element ne postavi na svoju ispravnu poziciju, dok Selection Sort izvršava samo jednu zamenu po prolazu, pomerajući najmanji (ili najveći) element direktno na njegovo ispravnu poziciju.



Programski kôd 1:

```

1  # Selection sort algoritam....
2
3  def selection_sort(array):
4      tekuci_idx = 0
5      while tekuci_idx < len(array) - 1:
6          najmanji_idx = tekuci_idx
7          for i in range(tekuci_idx + 1, len(array)):
8              if array[i] < array[najmanji_idx]:
9                  najmanji_idx = i
10
11         array[tekuci_idx], array[najmanji_idx] =
12         array[najmanji_idx], array[tekuci_idx]
13         tekuci_idx += 1
14
15     return array
16
17 arr = [9, 7, 5, 3, 1]
18 print(f"Ulaz: arr = {arr}")
19 print("Izlaz:", selection_sort(arr))
    
```

● Ulaz: arr = [9, 7, 5, 3, 1]
Izlaz: [1, 3, 5, 7, 9]

Selection Sort algoritam počinje sa spoljašnjom petljom koja predstavlja granicu između sortiranih i nesortiranih delova niza, u svakoj iteraciji pronalazi se najmanji element u nesortiranom delu i zamenjuje se sa indeksom tekućeg elementa. Promenljiva `tekuci_idx` prati trenutni indeks koji razdvaja sortirane i nesortirane delove niza. Promenljiva `najmanji_idx` je inicijalizovana na `tekuci_idx`, što predstavlja indeks

elementa pronađenog kao najmanji. Unutrašnja `for` petlja počinje od `current_idx+1` i iterira se kroz preostali nesortirani deo niza. Ona upoređuje svaki element sa elementom na `najmanji_idx` i ažurira `najmanji_idx` ako se pronađe manji element. Nakon nalaženja najmanjeg elementa, vrši se zamena elementa na `tekuci_idx` sa elementom na indeksu `najmanji_idx` `tekuci_idx` je inkrementiran i process se nastavlja dok se ne sortira ceo niz.

Jednostavniji kôd:

```
1  def selection_sort(arr):
2      n = len(arr)
3      for i in range(n):
4          min_idx = i
5          for j in range(i + 1, n):
6              if arr[j] < arr[min_idx]:
7                  min_idx = j
8          arr[i], arr[min_idx] = arr[min_idx], arr[i]
9      return arr
10
11 data = [64, 34, 25, 12, 22, 11, 90]
12 print(selection_sort(data.copy()))
```

Primer za vežbu:

Napisati Python kod za sortiranje tipa INSERTION, kao ulaz uneti broj elemenata niza slučajnih brojeva, kao i njihov opseg, gornju i donju granicu opsega.

Program treba da prikaže listu ulaznih random brojeva i sortiranu listu

```
1  import random
2
3  def insertion_sort(arr):
4      # Sortiranje arr in-place preko insertion sort
5      for i in range(1, len(arr)):
6          key = arr[i]          # element koji se ubacuje u sortirani deo niza
7          j = i - 1
8
9          # Pomeranje elemenata većih od key jednu poziciju desno
10         while j >= 0 and arr[j] > key:
11             arr[j + 1] = arr[j]
12             j -= 1
13
14         # Unos key na ispravnu poziciju
15         arr[j + 1] = key
16
17 # random input
18 n = 15          # broj elemenata niza arr
19 low = 10        # donja granica niza
20 high = 70       # gornja granica niza
21
22 # generisanje liste random brojeva [low, high]
23 data = [random.randint(low, high) for _ in range(n)]
24
25 print("Ulazna lista:", data)
26 insertion_sort(data)
27 print("Sortirana lista:", data)
28
```

Ulazna lista: [56, 15, 17, 44, 44, 11, 42, 42, 67, 22, 53, 67, 54, 28, 19]

Sortirana lista: [11, 15, 17, 19, 22, 28, 42, 42, 44, 44, 53, 54, 56, 67, 67]